Smart Environment Monitor

AI-powered air quality monitoring using ESP32 (AIoT) and Python

Theme	Artificial Intelligence of Things (AIoT) + AI Integrated Full Stack
Language	Python (backend, AI, dashboard) and Arduino C++ / MicroPython (ESP32)
Hardware	ESP32, DHT22, potentiometer (simulated on Wokwi)
Cost	Rs 0 - everything runs on free tools
Team name	_____
Team members	_____

1. Problem Statement

Air pollution and poor indoor air quality harm health, especially in cities like Bengaluru. Most people only learn the air is bad **after** they feel sick. Existing monitors only show current numbers; they do not predict problems or alert early.

2. Our Solution

A smart IoT device reads temperature, humidity and air quality, sends the data to a Python server, and an AI model detects abnormal readings and predicts upcoming air quality. A live dashboard shows graphs and sends a red alert when air becomes unsafe.

3. How It Works (System Flow)

Step	Stage	What happens
1	Sensors	DHT22 measures temperature and humidity. Potentiometer acts as the gas sensor (analog value).
2	ESP32	Reads the sensors every 5 seconds, converts analog to digital using its ADC, builds a JSON message.
3	WiFi / HTTP	ESP32 sends the JSON to the backend using an HTTP POST request.
4	FastAPI backend	Receives the data and stores it in an SQLite database.
5	AI model	scikit-learn (Isolation Forest) finds abnormal readings; a regression model predicts the next hours.
6	Streamlit dashboard	Live graphs, prediction, and a red alert when air quality is bad.

Flow in one line:

Sensor -> ESP32 -> WiFi -> FastAPI -> SQLite -> AI Model -> Streamlit Dashboard -> Alert

4. Hardware Components (Wokwi Simulator)

Wokwi (wokwi.com) is a free online simulator. We build and test the circuit virtually, so no hardware purchase is needed.

Component	Purpose	Real device replacement
ESP32 DevKit	Microcontroller with built-in WiFi. The brain of the system.	Same ESP32 board (about Rs 350-600)
DHT22	Measures temperature and humidity.	Same sensor
Potentiometer	Gives a changing analog value, acting as the gas sensor.	MQ135 air quality sensor
Wokwi project	Virtual breadboard, wires and serial monitor.	Real breadboard and jumper wires

5. Circuit Connections

Component pin	Connects to ESP32	Explanation
DHT22 VCC	3V3	Power supply for the sensor
DHT22 SDA (data)	GPIO 15	Digital data line
DHT22 GND	GND	Ground
Potentiometer left pin (GND)	GND	Ground
Potentiometer middle pin (SIG)	GPIO 34	ADC input pin (reads 0 to 4095)
Potentiometer right pin (VCC)	3V3	Power supply

Concepts to explain to judges (ECE)

- **ADC:** the ESP32 converts the analog voltage (0 to 3.3 V) into a digital value (0 to 4095, 12-bit).
- **GPIO:** general-purpose pins used to read sensors.
- **Why GPIO 34:** it is an input-only ADC pin, suitable for analog sensors.
- **Protocol:** the ESP32 uses WiFi and HTTP to talk to the backend.

6. ESP32 Program in Wokwi

Wokwi program type: **Arduino C++ (.ino)** on the ESP32 board. (MicroPython is also supported if the team prefers Python.) Add the **DHT sensor library for ESP32 (DHTesp)** in the Wokwi Library Manager.

```
#include <WiFi.h>
#include <HTTPClient.h>
#include "DHTesp.h"

const char* ssid = "Wokwi-GUEST";   // Wokwi's free virtual WiFi
const char* password = "";
const char* serverUrl = "https://YOUR-BACKEND-URL/data";

DHTesp dht;
const int DHT_PIN = 15;
const int GAS_PIN = 34;

void setup() {
  Serial.begin(115200);
  dht.setup(DHT_PIN, DHTesp::DHT22);
  WiFi.begin(ssid, password);
  while (WiFi.status() != WL_CONNECTED) delay(500);
  Serial.println("WiFi connected");
}

void loop() {
  TempAndHumidity d = dht.getTempAndHumidity();
  int gas = analogRead(GAS_PIN);

  String json = "{\"temp\":" + String(d.temperature) +
                ",\"humidity\":" + String(d.humidity) +
                ",\"gas\":" + String(gas) + "}";

  HTTPClient http;
  http.begin(serverUrl);
  http.addHeader("Content-Type", "application/json");
```

```
  int code = http.POST(json);
  Serial.println(json + " -> " + String(code));
  http.end();
  delay(5000);   // send every 5 seconds
}
```

Important: Wokwi runs in the cloud, so it cannot reach *localhost*. The backend must have a public URL (deploy on Render or Hugging Face Spaces, or use a free tunnel). Backup plan: the Python fake-sensor script below.

7. Python Software Part

Part	Tool	Job
Fake sensor (backup)	Python + requests	Generates random readings and sends them to the backend if Wokwi fails
Backend API	FastAPI	Receives data at /data and serves it to the dashboard
Database	SQLite	Stores every reading with a timestamp
AI model	scikit-learn	Isolation Forest for anomalies; regression for prediction
Dashboard	Streamlit	Live charts, prediction and red alert
Hosting	Streamlit Cloud / Render	Free public link for judges

Alert logic (simple and easy to explain)

- Gas value below 1500: **Good** (green).
- Gas value 1500 to 2500: **Moderate** (yellow).
- Gas value above 2500, or flagged as abnormal by the AI: **Danger** (red alert).
- These thresholds are examples; adjust them after testing.

8. Team Roles

Role	Responsibility	Name
Hardware lead (ECE)	Wokwi circuit, ESP32 code, explaining sensors and ADC	_____
Backend / AI	FastAPI, SQLite, machine learning model	_____
Frontend	Streamlit dashboard and alerts	_____
Presenter	Slides, pitch, demo flow	_____

9. Demo Plan (3 minutes)

- **0:00-0:30 Problem:** air pollution affects health and people find out too late.
- **0:30-1:00 Solution:** show the architecture slide (Sensor to Dashboard).
- **1:00-2:15 Live demo:** turn the potentiometer in Wokwi, watch the dashboard graph rise and the red alert appear.
- **2:15-2:45 AI:** show the anomaly detection and the prediction.
- **2:45-3:00 Impact and future:** homes, schools, factories, smart cities.

10. Impact and Future Scope

- Low cost: a real device costs under Rs 1,000.
- Early warning for homes, classrooms, hospitals and industries.
- Scalable: many devices can report to one dashboard.
- Next steps: real MQ135/PM2.5 sensors, a mobile app with SMS or WhatsApp alerts, and solar power.
- Replace the simulated ESP32 with a real one - the software stays the same.

11. Likely Judge Questions

Question	Short answer
Why simulate hardware?	To build and test quickly at no cost. The code is identical on a real ESP32; only the sensor is swapped.
Why Isolation Forest?	It detects unusual readings without needing labelled data, so it works with live sensor data.
How accurate is it?	Accuracy depends on real sensor data; in this prototype we demonstrate the full pipeline and will calibrate with real data.
What if WiFi fails?	The ESP32 can buffer readings and resend when the connection returns (future improvement).
Why ESP32?	Built-in WiFi, 12-bit ADC, low cost and low power.

Tools: Wokwi, Python, FastAPI, SQLite, scikit-learn, Streamlit, GitHub - all free.